



Università
degli Studi di
Messina

Università degli Studi di Messina

Dipartimento di Scienze Matematiche e Informatiche, Scienze Fisiche e
Scienze della Terra

Laboratorio di Reti e Sistemi Distribuiti

MapReduce

Wordcount di *The Lord of the Rings*

Alberto Paludetti
Matricola : 526650

Anno Accademico 2024/2025

Indice

1	Introduzione	2
2	Stato dell'Arte	3
3	Progettazione	4
4	Implementazione	5
4.1	Main	5
4.1.1	Redistribuzione Chunk Tra i Nodi	6
4.2	MapReduce.c	7
4.2.1	Gestisci_Ricezione	8
4.3	Client	11
4.3.1	Map	11
4.4	Reduce	13
4.5	MapReduce.h	14
5	Risultati e Test	16
5.0.1	Tempi Di Elaborazione dei Chunk	18
6	Conclusioni e Sviluppi Futuri	20

Capitolo 1

Introduzione

Il MapReduce è un algoritmo sviluppato da Google nel 2005 che permette di redistribuire il carico di lavoro tra più macchine all'interno di una rete. Abbiamo un *Master* che si occupa di spezzare i dati da elaborare in *chunk*, i quali in seguito vengono spediti ai *workers* che li elaborano. Questo algoritmo si compone di due fasi principali:

- **Map:** i dati vengono elaborati, e per ognuno di essi viene associato un valore numerico, creando così delle coppie chiave-valore.
- **Reduce:** questa fase viene eseguita dal Master, dopo che tutti i workers hanno completato la fase di elaborazione. Qui avviene un'aggregazione dei vari risultati che porta alla conclusione dell'algoritmo.

Il MapReduce è largamente utilizzato per gestire grandi quantità di dati in modo veloce e parallelo; infatti sta alla base di alcuni database NoSQL come, per esempio, MongoDB.

Capitolo 2

Stato dell'Arte

Negli ultimi decenni, con l'aumento esponenziale della quantità di dati generati e trattati da applicazioni web, scientifiche e aziendali, è emersa la necessità di sviluppare modelli di programmazione capaci di gestire in modo efficiente enormi volumi di informazioni.

Tra i primi approcci significativi troviamo l'elaborazione distribuita classica basata su cluster, in cui i dati vengono suddivisi e processati da più nodi connessi in rete. Questi sistemi, tuttavia, richiedevano una complessa gestione manuale delle comunicazioni, del bilanciamento del carico e della tolleranza ai guasti, rendendo difficile la loro adozione su larga scala.

Per semplificare questo paradigma, Google ha introdotto nel 2004 il modello **MapReduce**. L'idea di base è suddividere l'elaborazione in due fasi principali:

- **Map**, in cui i dati di input vengono trasformati in coppie chiave-valore;
- **Reduce**, in cui i valori associati a ciascuna chiave vengono aggregati per produrre il risultato finale.

L'efficacia di questo modello ha portato alla nascita di framework open-source come **Apache Hadoop**, che ha reso MapReduce accessibile al di fuori di Google. Hadoop ha introdotto un ecosistema completo per la gestione distribuita dei dati su cluster di macchine, includendo meccanismi di fault tolerance e il file system distribuito HDFS.

Successivamente, sono stati sviluppati modelli e framework alternativi che hanno migliorato o superato alcuni limiti di Hadoop. Un esempio è **Apache Spark**, che estende il paradigma di MapReduce introducendo la computazione in memoria, riducendo così la latenza e migliorando le prestazioni in scenari iterativi come l'analisi di grafi e il machine learning.

Oltre ai sistemi general-purpose, MapReduce è stato utilizzato come base per applicazioni specifiche, tra cui analisi testuale, motori di ricerca, data mining e bioinformatica. L'operazione di *wordcount*, ossia il conteggio delle occorrenze delle parole in un testo, è diventata un esempio canonico per illustrare il funzionamento del paradigma.

Capitolo 3

Progettazione

Per realizzare il progetto è stato creato un servizio basato su comunicazione TCP tra una macchina Master e tre macchine workers. Lo scambio di informazioni usufruendo dei servizi TCP è stato preferito alla controparte UDP per differenziare le connessioni con le macchine workers e per permettere una più efficace gestione dei pacchetti.

Il Master, inizialmente, si occupa di aprire e suddividere in chunk il file di testo contenente il primo libro del *Signore degli Anelli*, in seguito, si mette in ascolto sulla porta 8080 e aspetta che tutte le macchine workers si connettano. Questa è una fase fondamentale, per la cui realizzazione sono stati usati i concetti relativi ai thread e ai meccanismi di sincronizzazione studiati durante il corso. Ogni singolo thread si occupa di gestire la comunicazione con il client specifico; inoltre, prima di passare alla fase di Map, attraverso l'implementazione di un meccanismo di barriera, il programma attende che tutti e tre i nodi si siano connessi per cominciare. La comunicazione per ogni singolo client è identificata dal file descriptor generato dopo la chiamata `listen()` e passato al thread di gestione.

Capitolo 4

Implementazione

Per la realizzazione del progetto sono stati creati diversi file.

4.1 Main

Il main contiene una parte del codice eseguito dal nodo Master.

```
int main(){
    WordCount* risultati[MAX_CLIENT]; //mettiamo i valori di ritorno dei thread
    printf("ciao\n");
    printf("Inizio Programma\n");
    pthread_t thread[MAX_CLIENT]; //creiamo il pool di thread
    int contatore_thread=0;
    int numero_chunk=0;
    printf("numero chunk: %d\n",numero_chunk);
    char** Collezione_chunk=NULL; //Alloco lo spazio per contenere almeno un chunk
    chunk(&Collezione_chunk,&numero_chunk);
    printf("Prima di StampaChunk numero chunk: %d\n",numero_chunk);
    StampaChunk(Collezione_chunk,numero_chunk);
    //Dopo questa istruzione abbiamo il numero di chunk
    for(int i=0; i<numero_chunk;i++){
        printf("Chunk %d: %s\n",i,Collezione_chunk[i]);
    }

    //decidiamo qua come redistribuire i chunk tra i client.
    //impostiamo che il servizio per funzionare deve far si che si colleghino tutti i client, quindi M
    //redistribuiamo questi chunk su questi client
    int indice_Di_Redistribuzione=0; //questa variabile viene utilizzata prima della fase di creazione
    int Chunk_Per_Client=0;
    if(numero_chunk>MAX_CLIENT){
        if(numero_chunk%MAX_CLIENT==0){
            Chunk_Per_Client=numero_chunk/MAX_CLIENT;
        }else{
            Chunk_Per_Client=-1; //gestiamo in altro modo questa situazione
        }
    }
}
```

Figura 4.1: Main Inizio

La prima funzione che viene richiamata è **Chunk**, alla quale vengono passati gli argomenti **CollezioneChunk** e **NumeroChunk**. **CollezioneChunk** è un array di puntatori. Questa funzione apre il file del *Signore degli Anelli*, lo suddivide in pezzi (chunk), e per ognuno di essi aggiunge un puntatore che contiene l'indirizzo dell'area di memo-

ria del chunk generato. La funzione `chunk` utilizza un'altra funzione, `salvaChunk`, per effettuare l'effettivo allocamento dei chunk.

```
//alla fine del ciclo di letture del file, avremo un array di puntatori dinamico popolato dai vari chunk
void salva_chunk(char*** collezione_chunk, char* chunk, int *numero_chunk){
    printf("siamo in salva_chunk\n");
    char* copia= malloc(strlen(chunk)+1); // strlen restituisce il numero di caratteri visibili escludendo il terminatore di riga quindi poniamo +1
    // Non usiamo sizeof perchè ci restituirebbe la lunghezza del tipo, in questo caso il puntatore in un'architettura a 64 bit è 8 byte
    if (!copia) {
        perror("malloc fallita");
        exit(EXIT_FAILURE);
    }
    strcpy(copia, chunk); //copio il contenuto del chunk nel buffer copia, strcpy aggiunge in automatico il terminatore
    char **tmp = realloc(*collezione_chunk, (*numero_chunk + 1) * sizeof(char *)); //qui stiamo allocando lo spazio per un nuovo puntatore a char, infatti usiamo realloc
    if(!tmp){
        perror("errore in tmp");
        return;
    }
    *collezione_chunk=tmp; //aggiorno il puntatore originale
    (*collezione_chunk)[*numero_chunk] = copia; //inseriamo il chunk nel nuovo slot
    (*numero_chunk)++; //incrementiamo il numero dei chunk
}
```

Figura 4.2: SalvaChunk

4.1.1 Redistribuzione Chunk Tra i Nodi

La distribuzione dei chunk ai client è stata pensata per adattarsi a due modalità operative, a seconda del valore di `Chunk_Per_Client`.

Quando questo valore è diverso da -1, si adotta una distribuzione proporzionale: ogni client riceve lo stesso numero di chunk, memorizzati in una struttura `S_Chunk` che contiene anche il file descriptor della connessione. Per garantire che ciascun thread lavori su copie indipendenti, viene allocato dinamicamente un array di stringhe in cui vengono copiati i chunk prelevati dalla collezione originale. Una volta preparata la struttura, viene creato un thread dedicato che riceverà sia il file descriptor sia i dati da elaborare, evitando qualsiasi rischio di sovrascrittura tra thread. Il file descriptor viene usato per la comunicazione con quel determinato client.

Listing 4.1: Gestione della variabile `Chunk_Per_Client`

```
1 int Chunk_Per_Client=0;
2 if(numero_chunk>MAX_CLIENT){
3     if(numero_chunk%MAX_CLIENT==0){
4         Chunk_Per_Client=numero_chunk/MAX_CLIENT;
5     } else {
6         Chunk_Per_Client=-1;
7     }
8 }
```

```

printf("Siamo in Chunk Per_Client diverso da 1\n");
if(Chunk_Per_Client!=1){
    printf("Siamo in Chunk_Per_Client diverso da 1\n");
    //se numero chunk è diverso da meno 1 vuol dire che avremo una redistribuzione dei chunk in modo proporzionale tra i client
    //dobbiamo creare un array di S_chunk perchè nel momento in cui passiamo il successivo chunk al seguente thread, se non creo l'array di chunk vi sarà una sovrascrittura
    S_Chunk[indice_assegnazione_chunk].numero_chunk=Chunk_Per_Client;
    S_Chunk[indice_assegnazione_chunk].Array_Di_Chunk=malloc(numero_chunk*(sizeof(char*))); //alloco lo spazio per contenere i chunk
    S_Chunk[indice_assegnazione_chunk].fd=client_fd; //fd per la comunicazione col client
    for(int i=0;i<numero_chunk;i++){
        S_Chunk[indice_assegnazione_chunk].Array_Di_Chunk[i] = malloc(strlen(Collezione_chunk[indice_Di_Redistribuzione]) + 1); // +1 per '\0'
        strcpy(S_Chunk[indice_assegnazione_chunk].Array_Di_Chunk[i],Collezione_chunk[indice_Di_Redistribuzione]); //copiamo i chunk nella struttura da chunk, così potremo pas
    }
    pthread_create(&thread[contatore_thread],NULL,FunzioneThread,&S_Chunk[indice_assegnazione_chunk]); //Dobbiamo passare al thread sia l'FD della socket sia la struttura dat
    indice_assegnazione_chunk++; //dovrebbe arrivare a (MAX_CLIENT-1) .... Controllare
}else{
    //Entriamo qua se numero_chunk è -1, questo significa che vi sarà una distribuzione non proporzionale dei chunk tra i client connessi
    if(numero_chunk<MAX_CLIENT){
        S_Chunk[indice_assegnazione_chunk].numero_chunk=1; //un solo chunk per quasi tutti i client, vi saranno client con 0 chunk
        S_Chunk[indice_assegnazione_chunk].Array_Di_Chunk=malloc(1*(sizeof(char*))); //alloco lo spazio per un unico chunk
        S_Chunk[indice_assegnazione_chunk].fd=client_fd; //fd per la comunicazione col client
        strcpy(S_Chunk[indice_assegnazione_chunk].Array_Di_Chunk[0],Collezione_chunk[indice_Di_Redistribuzione]);
        indice_assegnazione_chunk++;
        //pthread_create
    }else{
        printf("Siamo in Chunk_Per_Client uguale a -1\n");
        //numero chunk maggiore di MAX_CLIENT
        int numero_di_chunk_da_assegnare_al_client=numero_chunk/MAX_CLIENT; //Esempio (11 chunk , 3 client)-> 11/3 = 3
        int resto=numero_chunk%MAX_CLIENT; //il resto lo assegnamo al primo chunk
    }
}

```

Figura 4.3: Gestione Chunk

4.2 MapReduce.c

In questo file troviamo tutte le funzioni che vengono eseguite dal Nodo Master e che permettono l'implementazione dell'algoritmo. Ogni thread esegue FunzioneThread.

```

void *FunzioneThread(void *arg) {
    WordCount ricevuto;
    Struttura_Chunk *mio_chunk = (Struttura_Chunk *)arg;
    //meccanismo di barriera
    pthread_mutex_lock(&mutex); //prendiamo possesso del lock
    variabile_condivisa++;
    if(variabile_condivisa<MAX_CLIENT){
        pthread_cond_wait(&cond,&mutex); //il thread si blocca e rilascia il mutex
    }else{
        pthread_cond_broadcast(&cond); //sveglia tutti i thread in attesa
    }
    pthread_mutex_unlock(&mutex);
    for (int i = 0; i < mio_chunk->numero_chunk; i++) {
        size_t len = strlen(mio_chunk->Array_Di_Chunk[i]);
        ssize_t sent = send(mio_chunk->fd, mio_chunk->Array_Di_Chunk[i], len, 0);
        if (sent < 0) {
            perror("Errore durante send");
            break;
        } else {
            printf("Inviato chunk %d: %zd byte\n", i, sent);
        }
    }
    printf("SIAMO DOPO IL FOR IN FUNZIONE THREAD, STIAMO PER INVOCARE GESTISCI RICEZIONE\n");
    WordCount* w=Gestisci_Ricezione(mio_chunk);
    printf("FD:%d,stringa: %s\n",mio_chunk->fd,w->parola);
    int chiusura=close(mio_chunk->fd);
    if(chiusura==1){
        perror("Errore nella chiusura di fd");
    }else{
        printf("Chiusura della connessione col client avvenuta col successore, FD->%d\n",mio_chunk->fd);
    }
    printf("Connessione chiusa col client\n");
    pthread_exit(w);
}

```

Figura 4.4: Funzione Thread

4.2.1 Gestisci_Ricezione

Questa è una funzione cruciale per il funzionamento dell'intero sistema. I client, dopo aver elaborato i dati eseguendo la funzione Map, li restituiscono al nodo Master usando la funzione `send()`. Per immagazzinare i dati che vengono spediti viene usata la funzione `recv()`, la quale restituisce il numero di byte letti.

Listing 4.2: Ricezione lunghezza parola da parte del Master

```
1 size_t ricevuti = 0;
2 while (ricevuti < sizeof(len_net)) {
3     int r = recv(mio_chunk->fd, ((char*)&len_net) + ricevuti,
4                 sizeof(len_net) - ricevuti, 0);
5     if (r <= 0) {
6         if (r == 0) printf("Connessione chiusa dal peer\n");
7         else perror("recv len_net");
8         goto fine;
9     }
10    ricevuti += r; // per ogni byte ricevuto incremento
    ricevuti
}
```

```
WordCount* Gestisci_Ricezione(Struttura_Chunk* mio_chunk){
    printf("siamo in gestisci Ricezione\n");
    WordCount* w = malloc(4 * sizeof(WordCount)); // spazio iniziale
    if (!w) {
        perror("malloc iniziale fallita");
        return NULL;
    }
    int indice = 0;
    int capienza = 4;

    while(1){
        int len_net;
        int cont_net;

        // Ricevi esattamente sizeof(len_net) byte per la lunghezza parola
        size_t ricevuti = 0;
        while (ricevuti < sizeof(len_net)) { //ricevuti deve essere minore di 4 byte, poichè len_net è un intero
            int r = recv(mio_chunk->fd, ((char*)&len_net) + ricevuti, sizeof(len_net) - ricevuti, 0);
            if (r <= 0) { //succede se il client chiude la connessione con closefd
                if (r == 0) printf("Connessione chiusa dal peer\n");
                else perror("recv len_net");
                goto fine;
            }
            ricevuti += r; //per ogni byte ricevuto incremento ricevuti
        }

        size_t len = (size_t) ntohs(len_net);
        printf("lunghezza della parola: %zu\n", len);
        if (len <= 0 || len > MAX_PAROLA) {
            fprintf(stderr, "Lunghezza parola non valida: %zu\n", len);
            goto fine;
        }

        char *parola = malloc(len);
        if (!parola) {
```

Figura 4.5: Gestione della ricezione dati

Tutti questi dati ricevuti vengono immagazzinati in una struttura dati di tipo `WordCount`, che permetterà in seguito di effettuare la fase di Reduce e di produrre il risultato finale.

Listing 4.3: Definizione della struttura `WordCount`

```
1 typedef struct {
2     char *parola;
3     int contatore;
4 } WordCount;
```

Funzione `Chunk`

Essa è la funzione responsabile della suddivisione in pezzi del file di testo del "Signore Degli Anelli". Apre attraverso la funzione `fopen` il file in sola lettura, alloca lo spazio per un buffer che conterrà una porzione dei caratteri del testo, l'ultimo carattere sarà sempre quello terminatore, senza di esso non potremmo usare le funzioni C di manipolazione delle stringhe. La suddivisione è stata implementata facendo in modo che nessuna parola venga spezzata, così da non ritrovarci parole troncate nei chunk. Tutto ciò è stato fatto usando una variabile chiamata `ultimo_spazio` che grazie alla funzione `strrchr`, immagazzina la posizione dell'ultimo spazio nel buffer. Successivamente, effettuando la differenza tra `ultimo_spazio` e buffer viene calcolata la lunghezza effettiva del chunk, il quale rispetto a prima sarà privato dell'ultimo parola poiché davanti all'ultimo spazio.

Listing 4.4: Funzione `Chunk`

```
1 char *ultimo_spazio = strrchr(buffer, ' '); //ci restituisce
    un puntatore a carattere che punta all'ultimo spazio nel
    buffer
2
    //se non ci sono
    spazi, cosa
    altamente
    improbabile,
    entriamo nell'
    if
3
4 if (!ultimo_spazio || ultimo_spazio==buffer) {
5     ultimo_spazio = buffer + n; //buffer punta al primo
    elemento dello spazio di memoria che abbiamo allocato,
6     //n sono i caratteri
    successivi che abbiamo
    immesso nel buffer,
7     //ultimo_spazio ora punta alla
    posizione successiva a
    tutto questo blocco
8 }
9
10 size_t lunghezza_chunk = ultimo_spazio - buffer;
11 printf("ultimo_spazio-  
buffer=%ld\n", ultimo_spazio -  
    buffer);
```

```

12
13 char *chunk = malloc(lunghezza_chunk + 1); //copiamo il chunk
    buono, senza parole spezzate
14 memcpy(chunk, buffer, lunghezza_chunk); //copiamo
    effettivamente i dati
15 chunk[lunghezza_chunk] = '\0'; //pongo il
    terminatore
16 //printf("chunk1: %s\n", chunk);
17 sleep(1);

```

Dopo questa fase, finalmente, il chunk può essere allocato al nostro insieme di chunk, rappresentato da `Collezione_Chunk`.

Listing 4.5: `Salva_chunk`

```

1 salva_chunk(collezione_chunk, chunk, numero_chunk);
2 printf("siamo_dopo_salva_chunk\n");
3 free(chunk); //libero lo spazio in memoria

```

La lettura del file viene effettuata tramite un puntatore che punta all'ultimo carattere letto. Tenendo conto di tutto ciò, nasce un nuovo problema. Con le operazioni precedenti abbiamo letto il file fino a un certo punto e salvato tutta quella porzione di caratteri nel buffer. In seguito, analizzando il buffer abbiamo tagliato l'ultima parola e creato il chunk finale. Quindi nel file dobbiamo tornare indietro dei caratteri che formano la parola tagliata, in modo tale da rileggerla e immetterla nel seguente chunk. Allora tramite il calcolo della differenza tra i caratteri letti e la lunghezza del chunk creato (escludendo l'ultima parola), è stato dato un valore alla variabile `indietro`, la quale viene utilizzata nella funzione `fseek` per riportare il puntatore al file a ritroso dei caratteri necessari.

Listing 4.6: `Riportare indietro il puntatore a file`

```

1 long indietro = n - lunghezza_chunk;
2 if (indietro > 0) {
3     fseek(f, -indietro, SEEK_CUR);
4 }

```

L'ultima porzione della funzione `chunk` si occupa di risolvere un problema presente nelle prime versioni del progetto: sistemare gli ultimi pezzi del testo letti dal file in modo da evitare che l'ultimo chunk contenga solo poche parole o una parola spezzata. In pratica, prende gli ultimi due chunk della collezione — il penultimo e l'ultimo — e calcola quanto spazio serve per unirli, aggiungendo uno spazio tra di loro e lo spazio per il terminatore `_0`.

Dopo aver ridimensionato il chunk penultimo con `realloc`, concatena prima uno spazio e poi l'ultimo chunk al suo interno. In questo modo, gli ultimi due chunk diventano un unico blocco coerente di testo. Infine, la funzione aggiorna la collezione dei chunk sostituendo il penultimo con la versione estesa, libera la memoria dell'ultimo chunk ormai inutile, decrementa il numero totale di chunk, libera il buffer temporaneo e chiude il file.

Listing 4.7: Unione Degli Ultimi due Chunk

```

1 //appena il file viene finito di leggere
2 char *penultimo = (*collezione_chunk)[(*numero_chunk) - 2];
3 char *ultimo = (*collezione_chunk)[(*numero_chunk) - 1];
4
5 size_t nuova_lunghezza = strlen(penultimo) + 1 + strlen(ultimo
6     ) + 1;
7
8 char *nuovo_penultimo = realloc(penultimo, nuova_lunghezza);
9 if (!nuovo_penultimo) {
10     perror("realloc fallita");
11     exit(EXIT_FAILURE);
12 }
13
14 strcat(nuovo_penultimo, "_");
15 strcat(nuovo_penultimo, ultimo);
16
17 (*collezione_chunk)[(*numero_chunk) - 2] = nuovo_penultimo;
18
19 free((*collezione_chunk)[(*numero_chunk) - 1]);
20
21 (*numero_chunk)--;
```

4.3 Client

I nodi lavoratori si connettono al nodo master tramite una socket TCP in cui sono specificati indirizzo IP e porta del server. Dopo lo stabilirsi della connessione, il client chiama la funzione `recv()` e attende che il server invii i chunk che dovranno essere elaborati.

4.3.1 Map

I client ,dopo aver ricevuto i dati da trattare, avviano la fase di Map. All'interno di questa sezione del codice, il programma si occupa di contare il numero di parole trovate. Per ogni nuova parola trovata viene aggiunta una struttura di tipo `WordCount` ad un'area di memoria allocata tramite `malloc` e successivamente modificata tramite `realloc`, in cui al suo interno viene salvata, tramite un puntatore di tipo `char`, la stringa e viene inizializzato il valore del contatore a uno. Se durante l'analisi del chunk verranno trovate, nuovamente, le stesse parole, verrà incrementato il valore del contatore legato al vocabolo ripetuto. La funzione `Map` utilizza, per verificare se una parola è già presente nel registro, la funzione `Controllo` che restituisce un intero:

- Valore 0: Se è la prima volta che quel vocabolo viene trovato
- Valore 1: Se il termine è già presente nel registro

Dopo aver passato in rassegna tutti i termini, la funzione Map restituisce una variabile di tipo Blocco_parole che contiene lunghezza_contatore il cui valore indica il numero di parole trovate, e il registro di tipo Word_count in cui al suo interno troviamo tutti i vocaboli con i contatori associati ad essi.

Listing 4.8: Map

```
1 Blocco_Parole Map(char* array) {
2     int capacita = INIZIALE;
3     WordCount *contatore_parole = malloc(capacita * sizeof(
4         WordCount));
5     if (!contatore_parole) {
6         perror("malloc fallita");
7         exit(EXIT_FAILURE);
8     }
9
10    int lunghezza_contatore = 0;
11    char buffer[100];
12
13    for (int i = 0; array[i] != '\0'; ) {
14        int j = 0;
15
16        // Salta tutti i caratteri non validi (spazi,
17        // punteggiatura)
18        while (array[i] != '\0' && !isalnum(array[i])) {
19            i++;
20        }
21
22        // Costruisci la parola con soli caratteri
23        // alfanumerici
24        while (array[i] != '\0' && isalnum(array[i])) {
25            buffer[j++] = tolower(array[i]);
26            i++;
27        }
28
29        buffer[j] = '\0';
30
31        // Evita parole vuote
32        if (j == 0) {
33            continue;
34        }
35
36        // Se la parola non è già presente
37        if (Controllo(buffer, contatore_parole,
38            lunghezza_contatore) == 0) {
39            if (lunghezza_contatore == capacita) {
40                capacita *= 2;
41                WordCount *tmp = realloc(contatore_parole,
42                    capacita * sizeof(WordCount));
43                if (!tmp) {
```

```

39         perror("realloc fallita");
40         exit(EXIT_FAILURE);
41     }
42     contatore\_parole = tmp;
43 }
44
45     char *copia = strdup(buffer);
46     if (!copia) {
47         perror("strdup fallita");
48         exit(EXIT_FAILURE);
49     }
50
51     contatore_parole[lunghezza_contatore].parola =
52         copia;
53     contatore_parole[lunghezza_contatore].contatore =
54         1;
55     lunghezza_contatore++;
56 }
57
58     Blocco_Parole blocco = { lunghezza_contatore,
59         contatore_parole };
60     return blocco;
61 }

```

4.4 Reduce

Dopo che tutti i nodi terminano la fase di **Map**, inviano al master i risultati. Esso li colleziona in un'unica area di memoria ed esegue la funzione **Reduce**. La funzione Reduce ha il compito di prendere i risultati parziali prodotti dai vari client e fonderli in un unico risultato complessivo. Nelle prime righe di codice, viene creata e inizializzata la variabile **totale** che conterrà tutte le parole e i contatori ad esse associate. Ha lo stesso ruolo del registro, trattato precedentemente, nella funzione **Map**. Viene effettuata una scansione dei risultati parziali dei singoli client, ogni nuova parola viene aggiunta a **totale**. Se una parola che arriva dai risultati parziali è già presente all'interno di **totale**, il suo contatore viene semplicemente aggiornato sommando la nuova occorrenza. In questo modo, parole uguali provenienti da client diversi vengono accorpate in un'unica voce. Inoltre, la funzione gestisce dinamicamente lo spazio a disposizione: se l'array non è più sufficiente, viene riallocato con una capacità maggiore per poter contenere ulteriori elementi. Alla fine dell'elaborazione, l'array viene ridimensionato alla dimensione effettiva e viene inserito un elemento con una parola nulla, che segnala la conclusione della lista. Il risultato finale restituito da Reduce è quindi una collezione compatta e completa di tutte le parole incontrate, ognuna accompagnata dal numero totale delle sue occorrenze.

Listing 4.9: Reduce

```

1 WordCount* Reduce(WordCount** risultati) {
2   printf("Siamo in reduce\n");
3
4   int capienza = 10;
5   int size = 0; // elementi usati
6   WordCount* totale = malloc(capienza * sizeof(WordCount));
7
8   for (int i = 0; i < MAX_CLIENT; i++) {
9       for (int j = 0; risultati[i][j].parola != NULL; j++) {
10          char* parola = risultati[i][j].parola;
11          int cont = risultati[i][j].contatore;
12
13          //cerchiamo se la parola si trova gi in totale
14          int trovata = 0;
15          for (int k = 0; k < size; k++) {
16              if (strcmp(totale[k].parola, parola) == 0) {
17                  totale[k].contatore += cont;
18                  trovata = 1;
19                  break;
20              }
21          }
22
23          //se la parola non stata trovata la aggiungiamo
24          if (!trovata) {
25              // Se serve, aumentiamo capacit
26              if (size >= capienza) {
27                  capienza += 10;
28                  totale = realloc(totale, capienza * sizeof(
29                      WordCount));
30              }
31              totale[size].parola = strdup(parola);
32              totale[size].contatore = cont;
33              size++;
34          }
35      }
36      totale = realloc(totale, (size + 1) * sizeof(WordCount));
37      totale[size].parola = NULL;
38      totale[size].contatore = 0;
39
40      return totale;
41  }

```

4.5 MapReduce.h

il file di intestazione del progetto contiene le strutture dati, le costanti e i prototipi di funzioni che utilizziamo nei vari file che sono stati implementati. Tra le costanti

troviamo:

- DIM_CHUNK : Rappresenta la grandezza, in byte, del chunk di dati.
- MAX_PAROLA : Lunghezza massima di una parola, quindi quanti dati può contenere il buffer.
- MAX_CLIENT : Numero di client che si dovranno connettere al server per far eseguire l'algoritmo.

Le strutture dati sono:

- Struttura_chunk: Contiene il file descriptor usato per comunicare con il client, il numero di chunk che il nodo dovrà trattare e l'array di stringhe in cui ogni stringa è un chunk.

```
1 typedef struct{
2     int fd;
3     int numero_chunk;
4     char ** Array_Di_Chunk;
5 } Struttura_Chunk;
```

- WordCount: è la struttura che contiene sia la parola sia il contatore legato ad essa

```
1 typedef struct {
2     char *parola;
3     int contatore;
4 } WordCount;
```

- Blocco_Parole: Struttura dati utilizzata dai nodi client per immagazzinare il registro delle parole trovate e per tenere traccia del numero di vocaboli contati.

```
1 typedef struct{
2     int lunghezza_contatore;
3     WordCount* struttura_parole;
4 }Blocco_Parole;
```

Capitolo 5

Risultati e Test

Il primo file che viene eseguito è quello relativo al nodo Master. Dopo la scomposizione del file in chunk, il server si mette in ascolto sulla porta 8080.

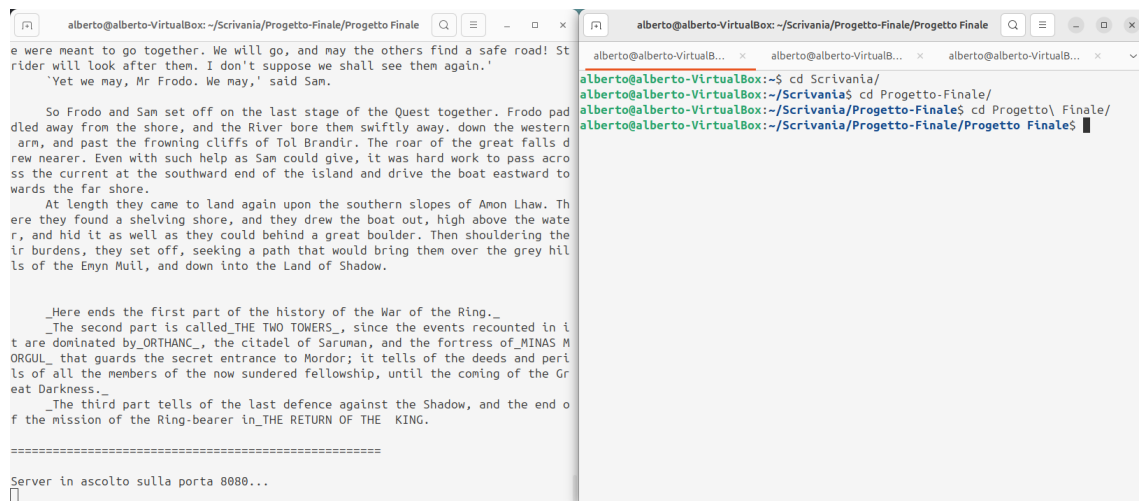
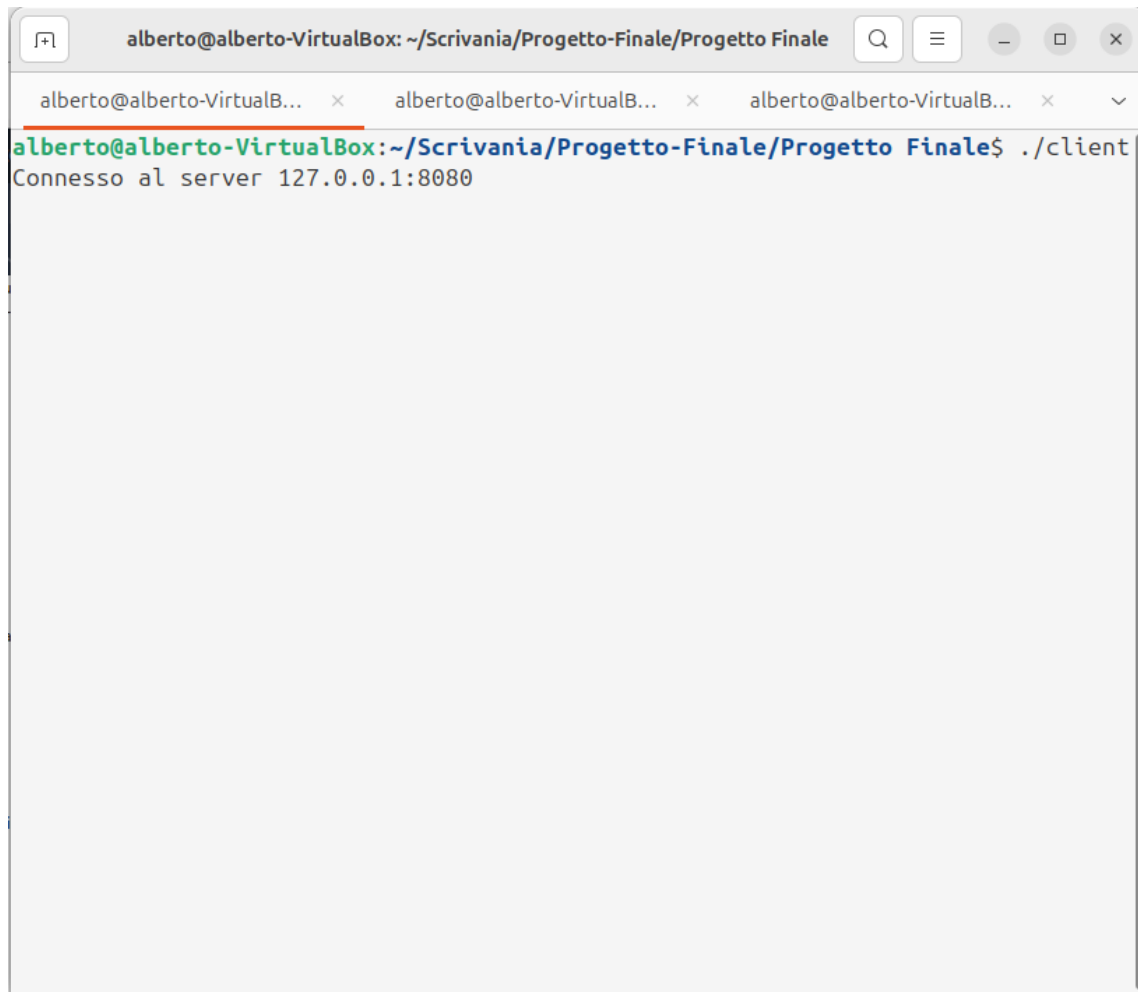


Figura 5.1: Inizio Esecuzione

Nel momento in cui avviamo i processi relativi ai client, essi si connettono al server, ma finché non sono tutti connessi il programma non va avanti, per via del meccanismo di barriera visto in precedenza.

A terminal window titled 'alberto@alberto-VirtualBox: ~/Scrivania/Progetto-Finale/Progetto Finale'. The prompt is 'alberto@alberto-VirtualBox:~/Scrivania/Progetto-Finale/Progetto Finale\$'. The command './client' has been entered, and the output is 'Connesso al server 127.0.0.1:8080'.

```
alberto@alberto-VirtualBox: ~/Scrivania/Progetto-Finale/Progetto Finale
alberto@alberto-VirtualB... x alberto@alberto-VirtualB... x alberto@alberto-VirtualB... x
alberto@alberto-VirtualBox:~/Scrivania/Progetto-Finale/Progetto Finale$ ./client
Connesso al server 127.0.0.1:8080
```

Figura 5.2: Connessione

Quando, finalmente, tutti i nodi sono connessi, il programma prosegue. Il nodo master invia i chunk ai nodi lavoratori e successivamente attende, mettendosi in modalità ricezione, dei risultati derivanti dall'elaborazione dei client.

```

Server in ascolto sulla porta 8080...
Connessione accettata da 127.0.0.1:35600
Abbiamo creato la struttura Chunk
Siamo in Chunk_Per_Client uguale a -1
siamo prima di strcpy
siamo prima di strcpy
siamo prima della pthread_create
Connessione accettata da 127.0.0.1:50368
Abbiamo creato la struttura Chunk
Siamo in Chunk_Per_Client uguale a -1
siamo prima della pthread_create
Connessione accettata da 127.0.0.1:44788
Abbiamo creato la struttura Chunk
Siamo in Chunk_Per_Client uguale a -1
siamo prima della pthread_create
siamo usciti dal ciclo
siamo nell'ultimo for quello del pthread_join
Inviato chunk 0: 238793 byte
SIAMO DOPO IL FOR IN FUNZIONE THREAD, STIAMO PER INVOCARE GESTISCI RICEZIONE
siamo in gestisci Ricezione
Inviato chunk 0: 262199 byte
Inviato chunk 1: 262193 byte
SIAMO DOPO IL FOR IN FUNZIONE THREAD, STIAMO PER INVOCARE GESTISCI RICEZIONE
siamo in gestisci Ricezione
Inviato chunk 0: 262198 byte
SIAMO DOPO IL FOR IN FUNZIONE THREAD, STIAMO PER INVOCARE GESTISCI RICEZIONE
siamo in gestisci Ricezione
lunghezza della parola: 7
lunghezza della parola: 5
lunghezza della parola: 7
lunghezza della parola: 4
lunghezza della parola: 5
lunghezza della parola: 5
lunghezza della parola: 5
lunghezza della parola: 3
lunghezza della parola: 4
lunghezza della parola: 6
lunghezza della parola: 4
lunghezza della parola: 5
lunghezza della parola: 6
lunghezza della parola: 4
lunghezza della parola: 4
lunghezza della parola: 5
lunghezza della parola: 6

```

Figura 5.3: Parte 2 Server

Il programma termina stampando l'intera lista delle parole trovate con i relativi contatori.

```

Fase di inizio Stampa dei risultati tra 5 secondi:
Parola: three, Contatore: 41
Parola: rings, Contatore: 24
Parola: for, Contatore: 548
Parola: the, Contatore: 4378
Parola: elven, Contatore: 27
Parola: kings, Contatore: 10
Parola: under, Contatore: 60
Parola: sky, Contatore: 18
Parola: seven, Contatore: 11
Parola: dwarf, Contatore: 31
Parola: lords, Contatore: 1
Parola: in, Contatore: 1264
Parola: their, Contatore: 244
Parola: halls, Contatore: 8
Parola: of, Contatore: 2144
Parola: stone, Contatore: 41
Parola: nine, Contatore: 24
Parola: mortal, Contatore: 3
Parola: men, Contatore: 24
Parola: doomed, Contatore: 1
Parola: to, Contatore: 1371
Parola: die, Contatore: 5
Parola: one, Contatore: 174
Parola: dark, Contatore: 109
Parola: lord, Contatore: 43
Parola: on, Contatore: 432
Parola: his, Contatore: 662
Parola: throne, Contatore: 2
Parola: land, Contatore: 60
Parola: mordor, Contatore: 28
Parola: where, Contatore: 86
Parola: shadows, Contatore: 29
Parola: lie, Contatore: 11
Parola: ring, Contatore: 132
Parola: rule, Contatore: 8
Parola: then, Contatore: 241
Parola: all, Contatore: 336
Parola: find, Contatore: 35

```

Figura 5.4: Risultato Finale

5.0.1 Tempi Di Elaborazione dei Chunk

Nelle seguenti immagini si possono notare le differenze temporali che vi sono tra il trattamento di un singolo chunk rispetto che a due, da parte di un client.

```
Chiusura della connessione col client avvenuta col successore, FD->5  
Connessione chiusa col client  
Tempo impiegato : 0.077870 secondi
```

Figura 5.5: Client che Elabora due chunk

```
Chiusura della connessione col client avvenuta col successore, FD->5  
Connessione chiusa col client  
Tempo impiegato : 0.016877 secondi
```

Figura 5.6: Client che Elabora un Chunk

Capitolo 6

Conclusioni e Sviluppi Futuri

Per sviluppare questo progetto sono stati fondamentali tutti i concetti appresi durante il corso. L'utilizzo dei thread ha permesso di suddividere efficacemente il lavoro tra i nodi della rete, i vari meccanismi di sincronizzazione usati (mutex, condition variables, barriere) sono stati di fondamentale importanza per l'implementazione dell'algoritmo, la creazione e gestione delle socket TCP ha permesso la comunicazione tra il nodo master e i nodi lavoratori. Questo lavoro, ha permesso di comprendere gli ingranaggi del MapReduce, inoltre mi ha fatto riconoscere lacune in concetti che pensavo di aver consolidato.